Studio Nx

백엔드 229 — 플러그인 스키마

2026-10-18

Backend(FastAPI/Python) 229 — 플러그인 스키마 (`ParamSchemaItem`)

항목	내용
파일	`studionx/backend/app/schemas/plugin.py`
작업	S1-09 기본 악기·이펙트: `param_schema` 항목과 플러그인 응답 (PRD 02 FR-241, FR-244, FR-246, FR-2187)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	228 `Track` Phase 2 · 230 디바이스 스키마

1. 하는 일

플러그인 API 가 주고받는 모양을 정합니다. 가장 중요한 것은 `ParamSchemaItem` 입니다. DB 의 `plugins.param_schema`(JSON) 안에 들어 있는 파라미터 한 개를 코드로 표현한 것으로, 화면이 이 값을 읽어 노브·슬라이더·선택 상자를 **자동으로 만듭니다** (FR-244·2187).

모델	하는 일
`ParamOption`	선택 상자(`select`)의 한 항목. `value`(코드가 쓰는 값)와 `label`(사람이 보는 이름)
`ParamSchemaItem`	파라미터 한 개. 이름, 종류, 범위, 기본값, 단위, 묶음(`group`), 핵심 여부(`primary`), 로그 곡선(`curve`)
`PluginResponse`	플러그인 한 건. `code`, `name`, `plugin_type`, `engine`, `version`, `param_schema`, `card_asset`, `is_active`
`PluginListResponse`	목록. `items` 만 담습니다 (총 개수는 플러그인이 많지 않아 넣지 않았습니다)
`PluginPresetResponse`	프리셋 한 건. `owner_id` 가 비어 있으면 시스템 프리셋입니다

2. 언제 쓰나

쓰는 곳	
`GET /plugins` · `GET /plugins/{id}` (237)	목록과 상세 응답을 이 모양으로 내보냅니다
`GET /plugins/{id}/presets` (237)	`PluginPresetResponse` 목록을 내보냅니다
화면의 카드·상세 모달	`primary` 인 항목 2~4개를 카드에, `group` 별로 나머지를 모달에 그립니다 (FR-2181·2185)
시드 SQL (242)	이 구조와 같은 JSON 을 `param_schema` 에 넣습니다

3. 코드

파라미터 한 개

```
class ParamSchemaItem(BaseModel):
    key: str
    name: str
```

```
    type: Literal["knob", "select", "toggle"] = "knob"
    group: str | None = None
    min: float | int | None = None
    max: float | int | None = None
    default: float | int | bool | str | None = None
    unit: str | None = None
    step: float | int | None = None
    curve: Literal["linear", "log"] | None = None
    primary: bool | None = None
    options: list[ParamOption] | None = None
```

플러그인 응답

```
class PluginResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    plugin_id: int
    code: str
    name: str
    plugin_type: PluginType
    engine: PluginEngine
    version: str
    param_schema: list[ParamSchemaItem]
    card_asset: str | None
    is_active: bool
```

문법 풀이

식	뜻
`Literal["knob", "select", "toggle"]`	셋 중 하나만 허용한다는 뜻입니다. 오타가 있으면 요청·응답 단계에서 걸립니다
`float | int | None`	소수, 정수, 없음 셋 다 됩니다. `cutoff` 는 소수, `pitch` 는 정수를 쓰기 때문입니다
`group: str | None = None`	값이 없어도 되는 칸입니다. 선택 항목은 기본값을 함께 적어 두면 빼도 됩니다
`ConfigDict(from_attributes=True)`	ORM 객체를 그대로 넣어도 읽어 준다는 설정입니다 (`PluginResponse.model_validate(plugin)`)
`default: ... | bool | str`	켜기/끄기와 선택 항목의 기본값까지 담으려고 넓혔습니다

4. 왜 이렇게

선택	이유
한 항목을 한 모델로	화면이 파라미터마다 따로 코드를 쓰지 않고, 이 구조를 돌면서 그립니다 (FR-244·2187)
`primary` 를 둠	카드에는 핵심 2~4개만 보여 줍니다. 어느 것이 핵심인지는 플러그인이 정합니다 (FR-2181)
`group` 을 둠	상세 모달에서 Osc 1, Filter 처럼 칸을 나눕니다 (FR-2185)
`curve` 를 둠	소리는 사람이 로그 눈금으로 듣습니다. `curve: "log"` 인 값(주파수)은 화면이 로그 노브로 그립니다
`type` 기본값을 `knob` 으로	대부분이 노브입니다. 시드의 47개 항목 중 41개가 종류를 적지 않아도 됩니다 (선택 4개, 켜기/끄기 2개만 적었습니다)
`Literal` 로 종류를 제한	`plugins.plugin_type` 등이 DB ENUM 과 같아야 합니다. 화면이 대소문자로 헷갈리지 않습니다

5. 시험

`tests/test_plugin_routes.py` 가 목록·상세·프리셋 응답의 칸 이름과 값(`code`, `param_schema` 항목의 `primary`, `owner_id` 구분)을 확인합니다. 시험은 시드와 겹치지 않도록 고유한 `code`(`pl_c_eq` 등)를 씁니다.